Programación para la administración de sistemas

Manual desarrollado para el alumnado de IFCT0609
Fundamentos y aplicaciones orientadas a UF1286
Edición de aula · Octubre de 2026

Qué vas a aprender

Programar consiste en describir una tarea con instrucciones suficientemente precisas para que un ordenador pueda ejecutarlas. En administración de sistemas esa tarea puede ser consultar el espacio libre, analizar un registro, inventariar una carpeta o generar un informe. Aprenderás a pasar de una necesidad expresada con palabras a un programa que recibe datos, los comprueba, realiza un cálculo y comunica el resultado.

El manual empieza desde los conceptos de programa, variable y algoritmo. Avanza hacia funciones, archivos, tratamiento de errores, procesos, memoria, pruebas y documentación. Un caso común, Aula Norte, conecta las explicaciones con situaciones de un centro formativo. Los ejemplos resueltos muestran el razonamiento y las actividades te piden modificarlo o aplicarlo a otros datos.

Alcance y relación con el certificado

Este material original se orienta a la UF1286, dentro de MF0964_3. Aporta una base de programación y desarrolla servicios y estructura del sistema operativo, lenguajes y bibliotecas, requisitos, ciclo de vida, calidad y optimización. Python es la herramienta principal de aprendizaje; C aparece como introducción a la memoria y al bajo nivel. Son elecciones didácticas, no una exigencia de lenguaje de la ficha.

No sustituye el desarrollo especializado de controladores de UF1287, las comunicaciones de UF1288 ni el diseño, integración y despliegue por componentes de MF0965_3. Se incluyen conexiones con esos temas para comprender el recorrido posterior. Las prácticas se realizan sobre datos ficticios y carpetas de laboratorio, con una cuenta normal.

Cómo utilizarlo

Lee cada explicación, predice qué hará el código y después ejecútalo. Copia primero el ejemplo sin cambios y modifica una sola cosa cada vez. Guarda las versiones que funcionan. Cuando aparezca un error, conserva su mensaje: es información para investigar. Los ejercicios finales se entregan sin solucionario; los ejemplos resueltos forman parte de la explicación.

Ruta de aprendizaje y preparación

Bloque	Contenidos
Comprender el sistema	Programa, proceso, servicio, recursos, núcleo y bibliotecas.
Construir programas	Algoritmos, variables, decisiones, bucles, colecciones y funciones.
Trabajar con datos reales	Rutas, archivos, CSV, JSON, excepciones e informes.
Administrar con criterio	Recursos, procesos, registros, memoria, pruebas y optimización.
Integrar lo aprendido	Proyecto de inventario de una carpeta, documentación y actividades.

Antes de comenzar

Necesitas un intérprete de Python 3 y un editor de texto o de código. Los ejemplos utilizan características disponibles en Python 3.10 o posterior y solo la biblioteca estándar. El profesor confirmará el entorno del aula. El nombre del ejecutable depende de la instalación: en muchos equipos Linux se usa python3; en Windows puede utilizarse py o python.

```
python3 --version
python3 ejemplo.py

# Alternativa habitual en Windows:
py --version
py ejemplo.py
```

Los símbolos del indicador de la terminal, como $ o >, no forman parte de estos comandos. No guardes programas con nombres como json.py, csv.py o subprocess.py: podrían ocultar los módulos de la biblioteca que vas a importar. Crea una carpeta llamada programacion_aula y trabaja dentro de ella.

Código completo y fragmentos

Un bloque con importaciones y datos de entrada suele poder guardarse como archivo independiente. Otros bloques continúan un ejemplo y se indica expresamente. La salida es el texto que aparece en la consola; no debes copiarlo dentro del programa. En Python, la sangría determina qué instrucciones pertenecen a un bloque. Usa cuatro espacios y mantén el mismo criterio en todo el archivo.

No hace falta entender todos los términos en la primera lectura. Anota qué crees que significa una palabra, comprueba su uso en el ejemplo y consulta el glosario al final. Una práctica está comprendida cuando puedes explicar sus entradas, sus decisiones y su resultado sin limitarte a leer el código.

1 Del problema al programa

Una necesidad concreta

Aula Norte quiere recibir un aviso cuando un disco tenga menos de un 15 % de espacio libre. La frase contiene una intención, pero aún faltan decisiones: qué disco se consulta, cómo se obtiene la medida, qué ocurre exactamente al alcanzar el 15 % y dónde se muestra el aviso. Programar exige resolver estas ambigüedades antes de escribir instrucciones.

Un algoritmo es una secuencia finita y ordenada de pasos para resolver un problema. Puede expresarse en lenguaje natural, con un diagrama o en pseudocódigo. Un programa es una representación ejecutable de esa solución en un lenguaje concreto. Dos programas escritos en lenguajes distintos pueden implementar el mismo algoritmo.

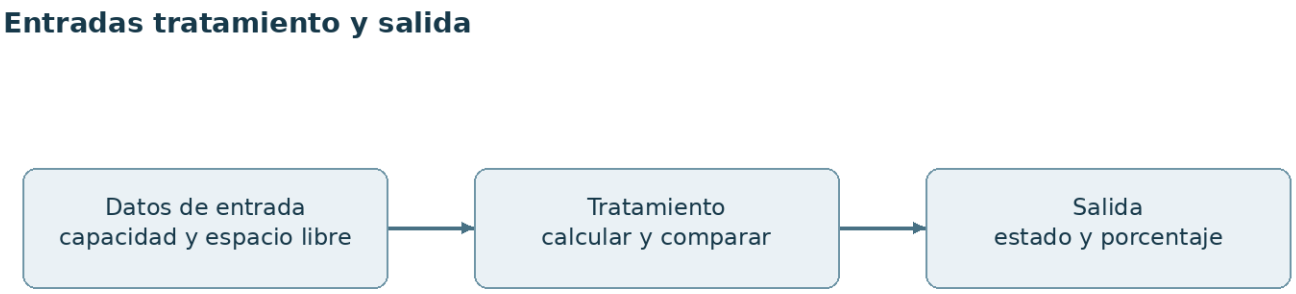


Figura 1. Separar las entradas, la transformación y el resultado ayuda a detectar requisitos incompletos.

Si el disco tiene 200 unidades de capacidad y 20 libres, el cálculo es 20 / 200 × 100 = 10 %. Como 10 es menor que 15, el estado será AVISO. Con 30 libres el resultado será 15 % y, según el requisito elegido, el estado será OK. Esta prueba de frontera evita confundir «menor que» con «menor o igual que».

Descripción inicial de la solución

Recibir capacidad y espacio libre; comprobar que la capacidad es positiva y que el espacio libre está entre cero y la capacidad; calcular el porcentaje; compararlo con el umbral; mostrar el resultado. Si la entrada no es válida, informar del error sin presentar una medida falsa como si fuera correcta.

Para practicar

Escribe las entradas y salidas de una herramienta que cuente archivos de una carpeta. ¿Debe contar subcarpetas? ¿Qué haría con un archivo que no puede leer? Plantea las preguntas antes de decidir la solución.

2 Programa proceso y servicio

Un archivo de programa guardado en el disco no está ejecutándose por el simple hecho de existir. Cuando se ejecuta, el sistema operativo crea un proceso con un identificador, memoria y recursos asociados. Se pueden ejecutar varias instancias del mismo programa: cada una puede tener sus propios datos y su propio estado.

Un servicio es una función ofrecida a otras aplicaciones o a usuarios. Puede apoyarse en uno o varios procesos. El servicio de matriculación depende de la aplicación, del acceso a la base de datos y de otros elementos. Que el proceso del servidor web exista no garantiza que una inscripción pueda guardarse. La comprobación debe reproducir la función relevante.

Concepto	Ejemplo en Aula Norte
Programa	Un archivo informe.py que contiene instrucciones.
Proceso	La ejecución concreta de informe.py en un equipo.
Servicio	La consulta de plazas o el acceso a archivos compartidos.
Recurso	Tiempo de CPU, RAM, almacenamiento, conexión o permiso.

Cómo colaboran los recursos

El procesador ejecuta instrucciones. La RAM sostiene datos y código que se están utilizando. El almacenamiento conserva archivos después de apagar. El sistema operativo coordina accesos y aplica permisos. La red interviene si la información debe viajar a otro equipo. Esta división es conceptual: una operación aparentemente sencilla suele necesitar varios de esos recursos.

Abrir un documento puede implicar leer sus bytes, reservar memoria, interpretar su formato y dibujarlo en pantalla. Guardarlo en una carpeta compartida añade comunicaciones y operaciones de escritura en otro equipo. Distintos fallos pueden producir un síntoma parecido, pero no cualquier fallo de cualquier componente genera el mismo efecto.

Ejemplo de diagnóstico razonado

Si el informe se guarda en una carpeta local pero no en la compartida, ya tienes una evidencia que orienta la investigación hacia el acceso remoto, sus permisos o su almacenamiento. No demuestra por sí sola qué elemento falla. Si tampoco puede guardarse localmente, ampliarías la revisión a la aplicación y al entorno del usuario.

Para practicar

Explica por qué «el servidor responde a una prueba de red» y «la matrícula se ha guardado correctamente» son evidencias distintas. Propón una prueba para cada afirmación.

3 El sistema operativo como intermediario

Los programas de usuario no deberían controlar libremente toda la memoria ni cualquier dispositivo. El sistema operativo establece una separación entre las aplicaciones y las funciones privilegiadas. Su núcleo gestiona mecanismos esenciales como planificación de procesos, protección de memoria y acceso a dispositivos. Esta separación ayuda a impedir que un error de una aplicación afecte sin control a las demás.

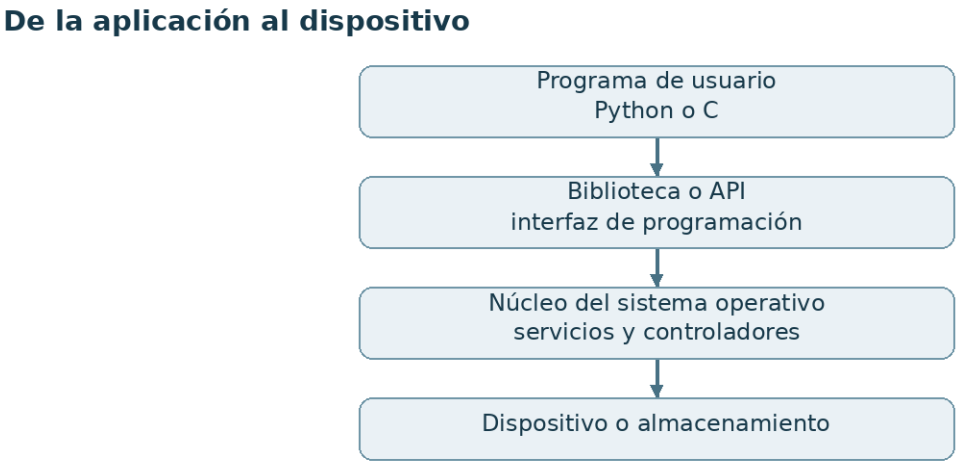


Figura 2. Esquema conceptual. Una operación de biblioteca puede implicar varias llamadas al sistema o resolverse sin ellas.

Una API es un conjunto de funciones y reglas que permite utilizar una capacidad sin conocer todos sus detalles internos. Una biblioteca reúne código reutilizable. Una llamada al sistema es una entrada controlada a un servicio del núcleo. Los conceptos se relacionan, pero no son sinónimos: una función de Python para calcular una media no necesita necesariamente pedir un servicio al núcleo.

Qué ocurre al abrir un archivo

El programa solicita abrir una ruta. Las capas de software traducen esa petición. El sistema comprueba la ruta y los permisos y, si procede, devuelve un recurso con el que leer o escribir. El programa debe gestionar que la operación falle: el archivo puede no existir, la cuenta puede carecer de permisos o el dispositivo puede estar desconectado.

En Python usamos bibliotecas como pathlib para manejar rutas y open para trabajar con contenido. No escribimos un controlador de disco cada vez que leemos un texto. Esta reutilización reduce errores, aunque sigue siendo necesario comprender los contratos de las funciones y liberar recursos cuando dejamos de necesitarlos.

4 Estructura y servicios del sistema operativo

Los sistemas operativos pueden organizarse de diferentes maneras. Estas arquitecturas describen decisiones sobre dónde se ejecutan sus funciones y cómo se comunican. No son etiquetas excluyentes: un sistema real puede combinar un núcleo con muchos servicios internos y la carga de módulos independientes.

Organización	Qué conviene comprender
Monolítica	Muchas funciones esenciales comparten el espacio del núcleo. La integración favorece comunicación directa, pero un fallo privilegiado puede tener gran impacto.
Modular	Permite incorporar o retirar determinadas piezas, como módulos del núcleo, mediante interfaces definidas. No significa ausencia de dependencias.
Microkernel	Mantiene un núcleo reducido y traslada servicios a procesos separados. La comunicación entre servicios adquiere gran importancia.
Por capas	Organiza responsabilidades sobre interfaces sucesivas para facilitar separación y comprensión. Es también una forma de diseñar otros sistemas.
Distribuida	Coordina funciones y recursos de varios equipos. Añade problemas de comunicación, fallos parciales y consistencia.

Memoria procesos archivos y dispositivos

La gestión de memoria proporciona espacios de direcciones y mecanismos de protección. La gestión de procesos decide cuándo pueden ejecutarse y cómo esperan por recursos. El sistema de archivos organiza nombres, directorios y metadatos. La entrada y salida permite transferir datos a dispositivos y redes. La seguridad decide qué identidad puede realizar cada operación.

Los registros del sistema aportan evidencias sobre sucesos; el arranque prepara las condiciones de funcionamiento; el apagado ordenado permite terminar actividades y completar operaciones pendientes. Una herramienta administrativa debe utilizar los servicios existentes en lugar de asumir que todos los recursos estarán siempre disponibles.

Virtualización y aislamiento

Una máquina virtual presenta hardware virtual a un sistema invitado. Un contenedor comparte el núcleo del anfitrión y aísla determinados recursos; no equivale a una máquina virtual completa. Ambos pueden servir para laboratorios, pero necesitan configuración y siguen utilizando recursos del equipo físico. Una instantánea facilita ciertos retrocesos, aunque no sustituye una política de copias recuperables.

Para practicar

Relaciona un fallo de permisos, un disco lleno y un proceso bloqueado con los servicios del sistema que investigarías. Describe el síntoma sin inventar la causa.

5 Elegir lenguaje y entorno de trabajo

El lenguaje se elige según la tarea, las interfaces disponibles y el nivel de control necesario. Python permite expresar con poco código operaciones administrativas y manejar datos estructurados. C facilita trabajar con memoria y bibliotecas de sistemas a un nivel más próximo a sus interfaces. Java o C# aportan ecosistemas de objetos y componentes. Un proyecto puede combinar varios lenguajes mediante interfaces bien definidas.

No conviene reducir la diferencia a «uno se compila y otro se interpreta». Existen compilación a código máquina, código intermedio e interpretación, y las implementaciones pueden mezclar técnicas. En el uso habitual de CPython, el código se compila a instrucciones intermedias que ejecuta el intérprete. En los ejemplos de C se genera un ejecutable para un entorno concreto.

Primer programa ejecutable

```
equipo = "AULA-PC01"
print("Revisión del equipo", equipo)
```

Guárdalo como saludo.py y ejecútalo desde la terminal situada en esa carpeta. La primera línea asocia el nombre equipo con un texto. La segunda utiliza print para mostrar un mensaje y el valor asociado. La salida será «Revisión del equipo AULA-PC01». Cambiar el nombre del archivo no cambia la variable: son elementos distintos.

Editor intérprete y terminal

El editor modifica texto. El intérprete ejecuta instrucciones Python. La terminal permite invocar programas y ver resultados. Un entorno de desarrollo puede reunir estas funciones, pero es útil distinguirlas. Si el editor encuentra el archivo y la terminal no, puede que cada uno esté trabajando desde una carpeta diferente.

Entornos virtuales

```
python3 -m venv .venv
# En Windows, según la instalación:
py -m venv .venv
```

Un entorno virtual separa dependencias de Python para un proyecto. No es una máquina virtual y no instala otro sistema operativo. Los ejemplos de este manual no requieren paquetes externos; si posteriormente se añaden, se debe registrar cuáles son y cómo reproducir la instalación. El profesor indicará cómo activar o seleccionar el intérprete del entorno en el aula.

Para practicar

Ejecuta saludo.py, cambia el código del equipo y vuelve a ejecutarlo. Anota qué archivo modificaste y desde qué carpeta lo invocaste.

6 Variables tipos y asignación

Una variable es un nombre que permite acceder a un valor durante la ejecución. En Python el tipo pertenece al objeto al que se refiere el nombre. Puedes volver a asociar el nombre a otro objeto, aunque hacerlo con significados incompatibles dificulta la lectura. Nombres como espacio_libre o numero_archivos explican mejor la intención que a o dato1.

Tipo	Ejemplo	Uso
int	archivos = 12	Cantidades enteras.
float	porcentaje = 12.5	Medidas aproximadas con decimales.
str	equipo = "AULA-PC01"	Texto entre comillas.
bool	disponible = True	Una condición verdadera o falsa.
NoneType	resultado = None	Ausencia de resultado, si el contrato lo permite.

Asignar no significa comparar

```
libres = 30
capacidad = 200
porcentaje = libres / capacidad * 100
print(porcentaje)
libres = libres - 5
print(libres)
```

La tercera línea calcula 15.0. La quinta consulta el valor anterior de libres, le resta 5 y asocia libres con 25. No es una ecuación matemática que afirme que un número es igual a sí mismo menos cinco. En pseudocódigo usaremos la flecha ← para asignar; en Python se utiliza =. La comparación de igualdad en Python se escribe ==.

Unidades y precisión

Un byte contiene ocho bits. Para expresar cantidades binarias, 1 KiB equivale a 1024 bytes y 1 GiB a 1024^3 bytes; GB se utiliza para 1000^3 bytes. No mezcles unidades al sumar o comparar. En este manual convertimos medidas de disco a GiB de manera explícita. float representa muchos decimales de forma aproximada; para dinero o requisitos exactos se necesita una estrategia adecuada, no confiar ciegamente en una igualdad decimal.

Para practicar

Calcula el porcentaje libre de una unidad con 500 GiB de capacidad y 125 GiB libres. Escribe variables con nombres claros y muestra el resultado con su unidad.

7 Operadores entrada y conversión

Los operadores combinan valores para obtener otros. En una expresión, los paréntesis permiten indicar la intención y evitan depender de una precedencia que el lector quizá no recuerde. Antes de ejecutar un cálculo, identifica los tipos: sumar dos textos no significa lo mismo que sumar dos números.

Familia	Python	Ejemplo y resultado
Aritmética	+ - * / // % **	17 // 5 es 3; 17 % 5 es 2.
Comparación	== != < <= > >=	10 < 15 produce True.
Lógica	and or not	True and False produce False.

La división / produce un resultado de tipo float para enteros. // realiza división redondeando hacia menos infinito; con positivos coincide con el cociente entero habitual. El resto % permite, por ejemplo, comprobar si un número positivo es par. No uses ^ para elevar a una potencia en Python: la potencia se escribe **.

Leer datos del usuario

```
texto = input("Número de equipos: ")
try:
    equipos = int(texto)
except ValueError:
    print("Introduce un número entero.")
else:
    if equipos <= 0:
        print("El número debe ser positivo.")
    else:
        print("Se revisarán", equipos, "equipos.")
```

input devuelve texto. int intenta interpretarlo como entero y puede fallar con ValueError. El programa distingue dos problemas: «tres» no tiene el formato esperado; -2 tiene formato entero, pero incumple la regla de cantidad positiva. Ambas comprobaciones son necesarias cuando la entrada proviene de una persona o de otro sistema.

El bloque else del try se ejecuta si la conversión no lanzó la excepción capturada. Esta organización evita usar equipos cuando la conversión no lo ha creado. En ejemplos iniciales con valores escritos en el código se presupone que tienen el tipo indicado; al introducir datos externos no debes mantener esa suposición.

Para practicar

Prueba 4, 0, -2 y el texto cuatro. Explica por qué no todos los errores son del mismo tipo. Añade un máximo de 30 equipos a la validación.

8 Decisiones y condiciones compuestas

Una decisión selecciona instrucciones según una condición. if evalúa una expresión; elif prueba otra condición si la anterior no se cumplió; else cubre el resto. En una cadena se ejecuta como máximo una de sus ramas. Varios if independientes pueden ejecutar varias ramas y por eso no siempre son intercambiables.

```
porcentaje = 12
if porcentaje < 5:
    estado = "CRITICO"
elif porcentaje < 15:
    estado = "AVISO"
else:
    estado = "OK"
print(estado)
```

Con 12 se muestra AVISO. No hace falta repetir porcentaje >= 5 en la segunda rama: haber llegado hasta ella implica que la primera condición era falsa. Con 5 se obtiene AVISO; con 15, OK. Escribir <= en lugar de < cambiaría el contrato en los límites.

Combinar condiciones

```
capacidad = 200
libres = 20
datos_validos = capacidad > 0 and 0 <= libres <= capacidad
if datos_validos:
    print(libres / capacidad * 100)
else:
    print("Medida inválida")
```

and exige que se cumplan ambas partes. or permite que baste una. not invierte el valor lógico. Python evalúa and y or con cortocircuito: puede dejar de evaluar cuando el resultado ya está decidido. Esto permite escribir capacidad > 0 and libres / capacidad < 0.15 sin dividir entre cero si la primera parte falla. Aun así, una validación separada puede resultar más clara para quien empieza.

Elegir pruebas antes de ejecutar

Para los umbrales 5 y 15, usa 4.9, 5, 14.9 y 15. Añade 0 y 100 si la entrada representa un porcentaje validado. Un valor negativo no debería clasificarse sin revisar primero su validez. Los mensajes al usuario deben diferenciar «no hay espacio» de «no se ha podido obtener la medida».

Para practicar

Diseña la clasificación de un tamaño de archivo: pequeño por debajo de 1 MiB, mediano desde 1 MiB hasta menos de 10 MiB y grande desde 10 MiB. Especifica si comparas bytes o MiB.

9 Repeticiones y trazas

Un bucle ejecuta un bloque varias veces. Se utiliza cuando la tarea se repite para varios elementos o hasta cumplir una condición. Elige la estructura según qué determina la repetición. Una traza es un registro manual del valor de las variables a cada paso; permite explicar el programa antes de ejecutarlo.

PARA y range en Python

```
total = 0
for numero in range(1, 5):
    total = total + numero
    print(numero, total)
```

range(1, 5) produce 1, 2, 3 y 4: el límite final queda excluido. En nuestro pseudocódigo, PARA i ← 1 HASTA 4 CON PASO 1 incluye el 4. La traducción debe tener en cuenta esa diferencia. En Python for recorre elementos de un iterable; no es solo un contador numérico.

Vuelta	numero	total antes	total después
1	1	0	1
2	2	1	3
3	3	3	6
4	4	6	10

MIENTRAS y repetición al menos una vez

```
intentos = 0
while intentos < 3:
    intentos += 1
    print("Intento", intentos)

while True:
    respuesta = input("¿Terminar? S/N: ").strip().upper()
    if respuesta == "S":
        break
```

El primer bucle comprueba antes y podría ejecutar cero vueltas con otra condición inicial. El segundo representa una repetición que termina al recibir S; Python no tiene una instrucción nativa REPETIR HASTA QUE. break sale del bucle más cercano. La condición de salida debe ser alcanzable y el programa debe avanzar hacia ella.

Para practicar

Predice range(2, 9, 2), range(5, 0, -1) y range(5, 1). Explica por qué no se modifica manualmente el contador del PARA didáctico. Cambia el acumulador para sumar solo números pares.

10 Colecciones y modelos de datos

Una variable aislada resulta incómoda cuando hay veinte equipos. Una colección reúne datos relacionados. La estructura adecuada depende de qué necesitas hacer: mantener orden, acceder por una clave, impedir duplicados o representar un registro con varios campos. Elegirla bien simplifica el resto del programa.

```
equipos = ["AULA-PC01", "AULA-PC02", "AULA-PC03"]
for nombre in equipos:
    print(nombre)

equipo = {"codigo": "AULA-PC01", "ram_gib": 8}
print(equipo["codigo"])
equipo["ram_gib"] = 16
```

Una lista mantiene una secuencia y permite acceso por posición. El primer índice es 0, no 1. Un diccionario asocia claves con valores. Acceder a una clave inexistente con corchetes lanza KeyError; get permite establecer un valor por defecto, pero no conviene usarlo para ocultar un campo obligatorio que falta. Un conjunto mantiene valores únicos y una tupla representa una secuencia que no se modifica directamente.

Referencias y cambios compartidos

```
original = [8, 16]
otra = original
otra.append(32)
print(original)  # [8, 16, 32]
copia = original.copy()
copia.append(64)
print(original)  # sigue siendo [8, 16, 32]
```

Asignar una lista a otro nombre no duplica automáticamente sus elementos: ambos nombres se refieren a la misma lista. copy crea una copia superficial, suficiente para esta lista de enteros. Con listas de diccionarios, los objetos internos seguirían compartiéndose. Hay que conocer el nivel que se copia antes de modificar registros complejos.

Un pequeño contrato de datos

Para el inventario acordamos codigo como texto no vacío y ram_gib como número positivo o un valor explícito de no verificado. No pongas 0 para representar desconocido si luego vas a calcular medias: estarías mezclando un dato real con ausencia de información. El formato necesita significado, unidades y reglas de validación.

Para practicar

Crea una lista con tres diccionarios de equipos ficticios. Recorre la lista y muestra los que tienen menos de 8 GiB. Decide cómo tratar una RAM que no se ha verificado.

11 Funciones contratos y alcance

Una función reúne instrucciones para una tarea con nombre. Sus parámetros reciben datos y su resultado se devuelve con return. Separar el cálculo de la lectura y la presentación permite reutilizarlo en una consola, una aplicación gráfica o una prueba automática. Una función pequeña facilita comprobar qué debe ocurrir ante cada entrada.

```
def porcentaje_libre(capacidad, libres):
    if capacidad <= 0 or not 0 <= libres <= capacidad:
        raise ValueError("Medida de disco inválida")
    return libres / capacidad * 100

resultado = porcentaje_libre(200, 30)
print(f"Libre: {resultado:.1f} %")
```

La salida es «Libre: 15.0 %». Los dos argumentos deben usar la misma unidad y ser números; el ejemplo valida su intervalo, no convierte texto. ValueError comunica que la función no puede proporcionar una medida válida. El código que la utiliza decidirá si muestra un mensaje, registra un error o solicita otros datos.

Devolver y mostrar son acciones distintas

print envía texto a una salida. return entrega un valor al código que llamó a la función. Si solo imprimes el porcentaje, otro cálculo no podrá utilizarlo directamente. Una función sin return explícito devuelve None. El nombre de una variable local tiene alcance dentro de la función y no debe confundirse con otro nombre igual definido fuera.

Contrato y documentación

Describe qué recibe, qué devuelve y qué errores puede producir. Para porcentaje_libre, el contrato exige capacidad positiva y libres entre cero y capacidad, en la misma unidad. El resultado se expresa de 0 a 100. No modifica el disco ni recibe una ruta. Separar esta función pura de la consulta al sistema hace más sencillo probarla con datos ficticios.

En un proyecto mayor, valida los límites en la capa adecuada y conserva la coherencia del contrato. No confíes en que toda función se usará únicamente desde el ejemplo que acabas de escribir: puede terminar en otra herramienta o recibir datos nuevos meses después.

Para practicar

Escribe una función clasificar_porcentaje que devuelva CRITICO, AVISO u OK. Reutilízala después de porcentaje_libre y prueba los límites de ambas funciones.

12 Módulos y separación de responsabilidades

Cuando crece el programa, un único archivo puede mezclar cálculo, lectura de datos y mensajes. Un módulo es un archivo Python importable. Una biblioteca proporciona módulos reutilizables; un paquete organiza módulos bajo un espacio de nombres. Separar archivos solo ayuda si también se separan responsabilidades y se definen interfaces claras.

```
# archivo medidas.py
def porcentaje_libre(capacidad, libres):
    if capacidad <= 0 or not 0 <= libres <= capacidad:
        raise ValueError("Medida inválida")
    return libres / capacidad * 100

# archivo principal.py, en la misma carpeta
from medidas import porcentaje_libre

def main():
    print(porcentaje_libre(200, 30))

if __name__ == "__main__":
    main()
```

Guarda los dos fragmentos en archivos separados con los nombres indicados. El bloque final hace que main se ejecute al invocar principal.py directamente. Al importarlo desde otro archivo, ese bloque no se ejecuta. Esto evita que una simple importación lance una lectura de teclado o una operación de escritura inesperada.

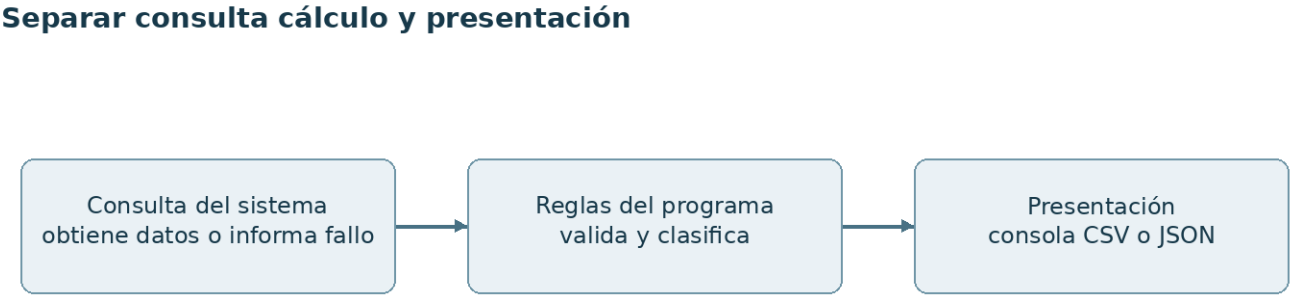


Figura 3. El cálculo puede probarse con datos inventados sin consultar un disco real.

Una interfaz define qué puede solicitar una parte a otra y qué resultado obtiene. La implementación explica cómo se consigue. Si la presentación solo necesita un número y un estado, no debería depender de detalles de un comando de Windows. Reducir esas dependencias permite sustituir una parte con menos cambios en el resto.

Para practicar

Propón tres módulos para una herramienta de inventario del aula. Escribe qué recibiría y devolvería cada uno. Evita nombres genéricos como cosas.py.

13 Rutas carpetas y permisos

Una ruta identifica una ubicación en un sistema de archivos. Puede ser absoluta, desde una raíz, o relativa al directorio de trabajo. La carpeta del archivo de programa y el directorio desde el que se ejecuta no tienen por qué coincidir. Esta diferencia explica muchos errores de «archivo no encontrado» aunque el documento exista.

```
from pathlib import Path

carpeta = Path("laboratorio")
archivo = carpeta / "informe.txt"
print("Directorio de trabajo:", Path.cwd())
print("Ruta calculada:", archivo.resolve())
print("Existe:", archivo.exists())
```

El operador / entre objetos Path construye una ruta; no divide números. resolve obtiene una ruta absoluta y resuelve componentes según su semántica; por sí solo no demuestra que el archivo exista. exists consulta su existencia en ese momento. Incluso después de una comprobación positiva, la operación posterior puede fallar si el archivo cambia, desaparece o pierde accesibilidad.

Permisos y enlaces

Una cuenta puede tener permiso para ver el nombre de un archivo y no para abrir su contenido. Los permisos efectivos dependen del sistema y de otros mecanismos, como listas de control de acceso. El programa debe intentar la operación y tratar el error esperado. No resuelvas un problema ejecutándolo todo como administrador: determina qué acceso requiere realmente.

Un enlace simbólico apunta a otra ruta. Si recorres carpetas siguiendo enlaces sin una política, podrías salir del alcance previsto o entrar en ciclos. En el proyecto final solo se examinan archivos inmediatos y se omiten enlaces simbólicos. Es una decisión de alcance didáctica, no una defensa suficiente ante un atacante que modifique rutas durante la ejecución.

Del nombre a la ubicación real

Una unidad física no equivale a una partición ni a una letra de unidad. Una carpeta compartida puede estar en otro equipo aunque aparezca integrada en el explorador. Para evaluar capacidad o permisos, identifica dónde se realiza la operación. Una ruta que se interpreta en Windows no siempre tiene el mismo sentido en Linux; Path utiliza la semántica del sistema donde se ejecuta.

Para practicar

Desde dos carpetas de trabajo distintas, calcula la ruta absoluta de laboratorio/informe.txt. Explica qué cambia. Hazlo solo con rutas de laboratorio, sin modificar carpetas del sistema.

14 Leer y escribir archivos de texto

Un archivo conserva una secuencia de bytes. Para trabajar con texto, un programa los interpreta mediante una codificación. UTF-8 permite representar muchos caracteres y es una elección habitual para intercambiar textos. Indicarla explícitamente evita depender de una configuración local que puede variar entre equipos.

```
from pathlib import Path

carpeta = Path("laboratorio")
carpeta.mkdir(exist_ok=True)
archivo = carpeta / "informe.txt"

with archivo.open("w", encoding="utf-8") as salida:
    salida.write("Equipo: AULA-PC01\n")
    salida.write("Estado: revisión realizada\n")

with archivo.open("r", encoding="utf-8") as entrada:
    for linea in entrada:
        print(linea.rstrip("\n"))
```

mkdir crea la carpeta y exist_ok permite que ya exista como directorio. El modo w crea el archivo o trunca su contenido si ya existe: úsalo solo sobre el archivo ficticio de esta práctica. El modo r lee. El modo a añade al final; el modo x crea un archivo nuevo y falla si ya existe. Elegir el modo es parte del requisito, no un detalle sin consecuencias.

Cerrar recursos incluso si hay un fallo

with utiliza un gestor de contexto que cierra el archivo al abandonar el bloque, también si se produce una excepción. Así no dependes de acordarte de close en todos los caminos. Cerrar una escritura permite completar operaciones pendientes en las capas habituales, pero no convierte por sí solo el archivo en una copia de seguridad resistente a cualquier fallo físico.

Leer por líneas o cargar todo

El bucle sobre entrada procesa una línea cada vez y evita cargar todo el texto de golpe. Leer todo puede ser sencillo para archivos pequeños, pero multiplica memoria y latencia con registros grandes. La decisión depende del tamaño, del tratamiento posterior y de si necesitas volver atrás. La optimización empieza por conocer estas necesidades.

Para practicar

Añade una tercera línea al archivo ficticio y cuenta cuántas líneas se leen. Compara el efecto de ejecutar dos veces con modo w y con modo a. Describe qué ocurre sin usar archivos reales.

15 Datos estructurados con CSV y JSON

Un texto libre permite escribir cualquier cosa, pero dificulta que otro programa identifique los campos. CSV representa registros tabulares. JSON representa valores, listas y objetos con claves. Ninguno garantiza por sí solo la calidad del dato: hace falta un esquema que defina campos obligatorios, tipos, unidades y reglas.

```
import csv
from pathlib import Path

ruta = Path("equipos_prueba.csv")
with ruta.open("w", newline="", encoding="utf-8") as f:
    escritor = csv.DictWriter(f, fieldnames=["codigo", "ram_gib"])
    escritor.writeheader()
    escritor.writerow({"codigo": "AULA-PC01", "ram_gib": 8})

with ruta.open(newline="", encoding="utf-8") as f:
    for fila in csv.DictReader(f):
        print(fila["codigo"], int(fila["ram_gib"]))
```

Este ejemplo escribe y sustituye equipos_prueba.csv en la carpeta de laboratorio. newline="" deja al módulo csv gestionar correctamente los finales de línea. No construyas CSV concatenando comas: los campos pueden contener comas, comillas o saltos. Al leer CSV los valores son texto y necesitan conversión y validación.

```
import json

equipo = {"codigo": "AULA-PC01", "ram_gib": 8}
texto = json.dumps(equipo, ensure_ascii=False, indent=2)
print(texto)
recuperado = json.loads(texto)
print(recuperado["ram_gib"])
```

dumps serializa un objeto a texto y loads interpreta el texto. JSON distingue números y cadenas, pero eso no prueba que ram_gib sea positiva. Un archivo bien formado puede contener datos inválidos para la aplicación. Evita formatos que ejecuten código al cargar datos no confiables; para el intercambio inicial utiliza un formato explícito y valida su contenido.

Para practicar

Diseña un esquema de equipo con codigo, sistema y ram_gib. Decide cómo representar un sistema no verificado. Prepara dos registros correctos y dos incorrectos y explica la regla que incumplen.

16 Excepciones y recuperación de errores

Un error de sintaxis impide interpretar el programa correctamente. Un error durante la ejecución puede producir una excepción. Un error lógico permite ejecutar, pero da un resultado incorrecto: por ejemplo, usar capacidad / libres al calcular el porcentaje. La estrategia de revisión cambia según el tipo de problema.

```
from pathlib import Path

ruta = Path("laboratorio") / "informe.txt"
try:
    texto = ruta.read_text(encoding="utf-8")
except FileNotFoundError:
    print("No existe el informe indicado.")
except PermissionError:
    print("La cuenta no puede leer ese informe.")
except UnicodeDecodeError:
    print("El contenido no se interpreta como UTF-8.")
except OSError as error:
    print("Fallo de acceso:", type(error).__name__)
else:
    print("Caracteres leídos:", len(texto))
```

Las excepciones específicas se colocan antes de una más general que también pudiera capturarlas. La variable texto solo se utiliza en else cuando la lectura terminó. El programa no imprime «correcto» después de un fallo ni devuelve un archivo vacío fingiendo que la lectura funcionó. Comunicar el estado real evita decisiones administrativas erróneas.

Qué capturar y dónde

Captura un error allí donde puedas recuperarte o comunicarlo con sentido. No rodees todo el programa con except sin tipo para ignorar cualquier problema: podrías ocultar un defecto propio o una interrupción del usuario. Tampoco presentes todos los fallos como «no existe» si en realidad puede faltar un permiso o haberse desconectado una unidad.

Diagnosticar con evidencias

Lee el mensaje y localiza la línea. Reproduce el problema con datos mínimos. Comprueba ruta, directorio de trabajo y condiciones de entrada. Cambia una sola causa posible y vuelve a ejecutar. Si una operación es destructiva, detenerse puede ser la respuesta correcta; continuar con datos incompletos necesita una política explícita, como hará nuestro inventario con archivos inaccesibles.

Para practicar

Ejecuta la lectura con un archivo ficticio presente y con una ruta inexistente. Anota el resultado. No cambies permisos de carpetas del sistema para provocar un error: el profesor puede preparar un caso controlado.

17 Consultar recursos de almacenamiento

Una herramienta administrativa debe diferenciar la medida obtenida de su interpretación. La biblioteca consulta el sistema; una función de reglas clasifica el resultado. Esa separación permite probar umbrales sin llenar discos ni modificar cuotas. Las medidas de espacio son instantáneas y pueden cambiar mientras otros procesos escriben.

```
import shutil
from pathlib import Path

uso = shutil.disk_usage(Path.cwd())
gib = 1024 ** 3
porcentaje = uso.free / uso.total * 100
print(f"Capacidad: {uso.total / gib:.2f} GiB")
print(f"Libre: {uso.free / gib:.2f} GiB")
print(f"Porcentaje libre: {porcentaje:.1f} %")
if porcentaje < 15:
    print("AVISO: poco espacio disponible")
else:
    print("OK según el umbral de esta práctica")
```

disk_usage devuelve total, usado y libre en bytes para el sistema de archivos correspondiente a la ruta. No descubre por sí solo todos los discos físicos del equipo ni identifica cada partición. Si consultas una carpeta situada en otra unidad montada, puedes obtener otra medida. Las cuotas, reservas y permisos pueden hacer que el usuario no pueda escribir todo el espacio que parece libre.

Una muestra no es monitorización continua

Un dato aislado describe un momento. Para monitorizar se necesita decidir periodicidad, almacenamiento de muestras, umbrales y respuesta. No consultes cada milisegundo si el problema cambia lentamente: tu herramienta también consume recursos. Evita disparar un aviso distinto cada segundo por el mismo problema; define cuándo aparece, cuándo se considera resuelto y quién lo recibe.

Requisito y prueba

«Mostrar el porcentaje libre de la unidad consultada con una cifra decimal» es verificable. «Que el disco vaya bien» no lo es. Comprueba el cálculo con 100 de capacidad y 14 libres, con 15 y con 16. Después verifica que el programa obtiene una medida del sistema y muestra sus unidades. El dato real puede variar, así que no fijes un valor idéntico para todos los equipos.

Para practicar

Separa el cálculo y la clasificación en funciones. Diseña un registro con fecha, ruta consultada y porcentaje. No programes aún tareas periódicas: primero comprueba una ejecución manual.

18 Lanzar procesos y recoger resultados

A veces una función solo está disponible mediante otro programa. Ejecutarlo desde nuestro código crea un proceso hijo. Es necesario conocer su contrato: argumentos, salida, códigos de retorno y tiempo máximo de espera. Un proceso puede arrancar bien y terminar con error; también puede bloquearse esperando una entrada que no hemos proporcionado.

```
import subprocess
import sys

try:
    resultado = subprocess.run(
        [sys.executable, "-c", "print(2 + 3)"],
        capture_output=True, text=True,
        timeout=5, check=True
    )
except subprocess.TimeoutExpired:
    print("Se agotó el tiempo de espera.")
except subprocess.CalledProcessError as error:
    print("El proceso falló:", error.returncode)
except OSError as error:
    print("No se pudo iniciar:", type(error).__name__)
else:
    print(resultado.stdout.strip())
```

El proceso hijo usa el mismo intérprete que está ejecutando el programa y escribe 5. stdout es su salida normal; stderr corresponde a mensajes de error. check=True transforma un retorno no cero en CalledProcessError. timeout limita la espera de la ejecución, aunque el inicio del proceso puede tener limitaciones propias de la plataforma. No se ejecuta un comando elegido por un usuario.

Argumentos y shell

La lista separa el programa de sus argumentos. shell=False es el valor predeterminado y evita interpretar la orden mediante una shell. Concatenar entradas de usuario dentro de una cadena que se envía a shell=True puede permitir ejecutar operaciones no previstas. Incluso sin shell, debes validar qué programa y opciones autorizas; una lista no vuelve inocuo cualquier comando.

Portabilidad y recursos

Los nombres y opciones de comandos administrativos difieren entre Windows y Linux. Prefiere una API portable si satisface la necesidad y utiliza adaptadores específicos cuando no. capture_output conserva la salida en memoria: no es adecuada para una salida ilimitada. Diseña lectura incremental o límites si esperas grandes volúmenes.

Para practicar

Cambia únicamente el código fijo del hijo por print(7 * 8). Después describe cómo distinguirías «no arrancó», «terminó con error» y «tardó demasiado», sin matar procesos ajenos.

19 Registros y trazabilidad

Un registro de eventos permite reconstruir qué ocurrió. Un mensaje útil incluye fecha, operación, nivel y contexto suficiente, como el código del equipo o el número de archivos procesados. No debe incluir contraseñas, claves ni datos personales innecesarios. Registrar todo sin criterio aumenta ruido y ocupa almacenamiento.

```
import logging

logging.basicConfig(
    filename="actividad_prueba.log",
    encoding="utf-8", level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s"
)
logging.info("Inicio de revisión del laboratorio")
archivos = 3
logging.info("Archivos revisados: %d", archivos)
logging.warning("Ejemplo ficticio de aviso")
```

La práctica añade registros al archivo actividad_prueba.log del directorio de trabajo. INFO describe hechos normales, WARNING señala una situación que merece atención y ERROR un fallo de una operación. DEBUG se reserva habitualmente para diagnóstico detallado. Los niveles son una convención que el equipo debe aplicar de forma coherente.

Un informe y un registro no cumplen el mismo papel

El informe resume el resultado para una persona o para otra aplicación. El registro documenta eventos durante el proceso. Un informe puede indicar «12 archivos inventariados, 1 omitido». El registro puede explicar que la omisión se debió a un error de permisos en un momento concreto. La ruta completa podría revelar nombres personales; decide si es realmente necesario conservarla.

Límites y mantenimiento

En una herramienta real se define cuánto se conserva, quién accede, cuánto puede crecer y cómo rota el registro. Una cuenta sin permiso para escribirlo puede impedir registrar; la política del programa debe decidir si detenerse o avisar por otro canal. Tampoco un archivo de registro demuestra automáticamente que una operación produjo el efecto esperado: su contenido debe apoyarse en comprobaciones.

Para practicar

Redacta tres mensajes: inicio de inventario, fallo al consultar un archivo y finalización con omisiones. Indica nivel y datos que excluirías. Explica qué necesitarías para no confundir dos ejecuciones distintas.

20 Memoria y eficiencia al procesar datos

La memoria virtual ofrece a los procesos una visión de direcciones que el sistema relaciona con memoria física y otros mecanismos. No significa que el almacenamiento se vuelva tan rápido como la RAM. Si una carga requiere mover continuamente páginas entre RAM y almacenamiento, el rendimiento puede empeorar mucho. Una herramienta debe medir y reducir su uso de memoria antes de culpar al equipo.



Figura 4. La jerarquía resume funciones y latencias relativas, no capacidades ni tiempos universales. La memoria virtual es un mecanismo de gestión, no otro disco físico.

Procesar por partes

Para contar errores en un registro de varios gigabytes no necesitas cargarlo entero. Puedes leer una línea, actualizar un contador y descartarla antes de leer la siguiente. La memoria necesaria dependerá de la línea más grande y de lo que conserves, en vez de crecer con todo el archivo. Si además guardas todas las líneas en una lista, pierdes esa ventaja.

```
from pathlib import Path

contador = 0
with Path("actividad_prueba.log").open(encoding="utf-8") as f:
    for linea in f:
        if " WARNING " in linea:
            contador += 1
print("Avisos:", contador)
```

Este ejemplo analiza solo el formato sencillo del registro anterior. Un analizador de producción necesita un formato estable y reglas para líneas incompletas o versiones distintas. El nombre WARNING dentro de un mensaje podría confundir una búsqueda ingenua; al ampliar el formato convendría analizar campos en lugar de buscar subcadenas sin contexto.

Para practicar

Compara conceptualmente este recorrido con read_text().splitlines(). ¿Qué información se conserva en cada caso? ¿Qué ocurre si una única línea tiene un tamaño desmesurado?

21 Introducción a C y a los punteros

C permite ver conceptos que un lenguaje de alto nivel suele gestionar por nosotros: tamaños de representación, direcciones y límites de memoria. Aprender estos conceptos ayuda a entender llamadas al sistema, bibliotecas y controladores. Esta introducción se ejecuta como programa de usuario; no instala módulos ni modifica el núcleo.

```
#include <stdio.h>

int main(void) {
    int cantidad = 3;
    int *direccion = &cantidad;
    printf("Cantidad: %d\n", cantidad);
    *direccion = 4;
    printf("Ahora: %d\n", cantidad);
    return 0;
}
```

cantidad contiene un entero. &cantidad obtiene su dirección. direccion es un puntero a int: guarda una dirección que se interpreta como ubicación de un entero válido. *direccion accede al objeto apuntado. La asignación cambia cantidad y la salida muestra primero 3 y después 4. Un puntero no es una garantía automática de que la dirección siga siendo válida.

Compilar en el laboratorio

```
cc -std=c11 -Wall -Wextra ejemplo.c -o ejemplo
./ejemplo
```

Estos comandos presuponen un compilador C disponible en el entorno Linux del laboratorio. Si no existe, consulta al profesor; no hace falta instalarlo para leer el ejemplo. -Wall y -Wextra activan advertencias útiles. Compilar sin errores no demuestra que el programa sea correcto; hay fallos de memoria que aparecen solo al ejecutar o con entradas determinadas.

Límites y duración de los objetos

Un array de tres enteros tiene índices 0, 1 y 2; acceder al índice 3 queda fuera de sus límites. Un objeto local deja de existir cuando termina su ámbito; devolver la dirección de una variable local no proporciona almacenamiento válido para usarla después. La memoria dinámica obtenida con malloc necesita comprobar fallos y liberarse con free cuando corresponda. Ni acceder a memoria liberada ni liberarla dos veces es válido.

En Python los objetos y sus referencias se gestionan automáticamente, pero se pueden retener grandes estructuras por descuido. En C el control adicional exige disciplina explícita. La elección del lenguaje depende del problema y de las interfaces que el sistema ofrezca, no de que uno sea universalmente mejor.

Para practicar

Dibuja cantidad y direccion como dos celdas distintas. Señala qué contiene cada una y qué cambia al ejecutar *direccion = 4. No inventes una dirección numérica real.

22 Concurrencia y recursos compartidos

Dos tareas concurrentes pueden progresar en periodos de tiempo que se solapan. Eso no implica que se ejecuten exactamente al mismo instante: el paralelismo depende del hardware y del entorno. Un proceso tiene recursos propios; un hilo es una unidad de ejecución dentro de un proceso y suele compartir memoria con otros hilos del mismo proceso.

Si dos tareas leen un contador que vale 5, ambas calculan 6 y ambas escriben 6, se pierde un incremento. El problema no está en sumar mal, sino en realizar una operación compuesta sin coordinación. Este tipo de resultado dependiente del orden de ejecución se conoce como condición de carrera.

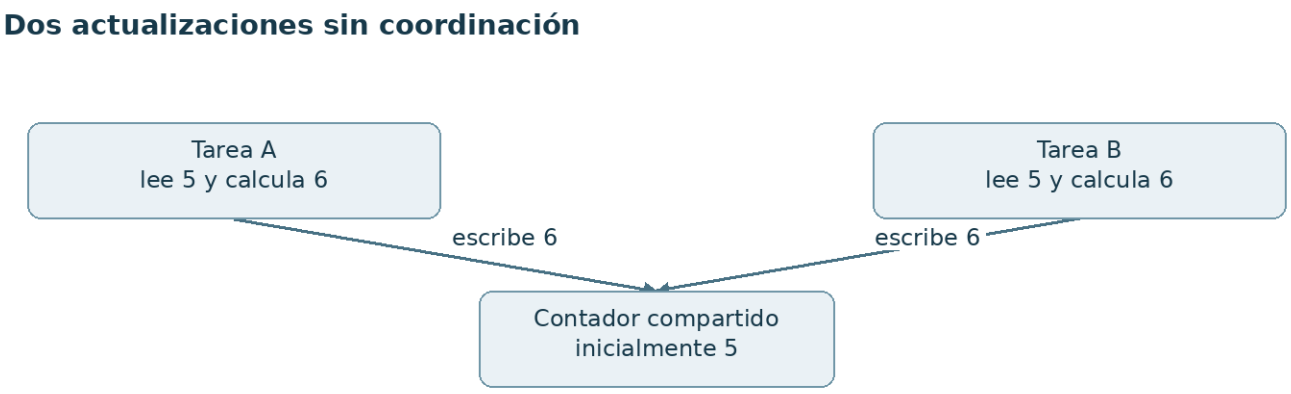


Figura 5. El resultado puede ser 6 cuando se esperaban dos incrementos y un valor final de 7.

Coordinar antes de añadir tareas

Un bloqueo puede proteger una sección crítica para que solo una tarea la ejecute a la vez. Una cola permite intercambiar trabajo sin que cada consumidor manipule directamente todos los datos compartidos. Pero bloquear demasiado reduce concurrencia y adquirir varios bloqueos en órdenes incompatibles puede producir un interbloqueo: las tareas esperan unas por otras indefinidamente.

Para una primera herramienta de inventario, un recorrido secuencial suele ser más comprensible. Añadir hilos sin medir puede dificultar el diagnóstico y aumentar la presión sobre el almacenamiento. Una copia de archivo tampoco es atómica solo porque se ejecute desde un hilo distinto. Hay que decidir qué coherencia se necesita y cómo se confirma que terminó.

Para practicar

Describe qué ocurriría si dos ejecuciones escriben el mismo informe a la vez. Propón una política de nombres o coordinación. Explica por qué una comprobación inicial de existencia no elimina todas las carreras.

23 Analizar requisitos y diseñar la solución

Antes de construir una utilidad, define quién la utilizará y qué problema resolverá. Un requisito funcional describe una capacidad: inventariar archivos inmediatos de una carpeta. Un requisito de calidad establece cómo debe comportarse: no modificar los originales, informar de omisiones o responder en un tiempo acordado para una carga definida.

Requisitos del proyecto Aula Norte

Identificador	Criterio verificable
R1	Recibir una carpeta y una ruta de informe desde la línea de comandos.
R2	Inventariar únicamente archivos regulares inmediatos; omitir subcarpetas y enlaces.
R3	Recoger nombre y tamaño en bytes, con orden alfabético reproducible.
R4	Crear un CSV UTF-8 sin sobrescribir un archivo existente.
R5	No modificar los originales; informar de errores y del número de omisiones.
R6	Devolver un código que distinga éxito, fallo y finalización parcial.

La herramienta no garantiza una fotografía atómica de una carpeta activa: los archivos pueden cambiar durante la revisión. Tampoco descubre todos los componentes del ordenador ni calcula espacio libre de disco. Esas funciones serían requisitos distintos. Limitar el alcance permite decidir qué significa «correcto» y qué queda fuera de esta versión.

Diseñar datos y flujo

El informe tendrá columnas nombre y bytes. La función de inventario devolverá filas y un contador de omisiones. El programa principal recibirá las rutas, llamará a la función, escribirá el informe y mostrará el resumen. Una decisión de diseño importante será guardar el CSV fuera de la carpeta de entrada para que no se incluya a sí mismo.

Un diagrama de flujo representa decisiones y repetición. Un modelo de datos describe campos y tipos. Un diagrama de componentes presenta responsabilidades y dependencias. No son dibujos intercambiables: cada uno responde a una pregunta. Hazlos lo bastante sencillos para poder contrastarlos con los requisitos.

Para practicar

Añade un requisito para filtrar por extensión y define tres pruebas de aceptación. Decide si .TXT y .txt se tratarán igual y deja esa decisión por escrito.

24 Ciclo de vida y calidad del software

Un programa administrativo también necesita un ciclo de vida: comprender la necesidad, diseñar, implementar, probar, desplegar, mantener y retirar cuando deja de ser útil. No siempre se avanza de forma lineal. Una prueba puede descubrir que falta un requisito y obligar a revisar el diseño. Trabajar por incrementos permite comprobar pequeñas capacidades antes de acumular complejidad.

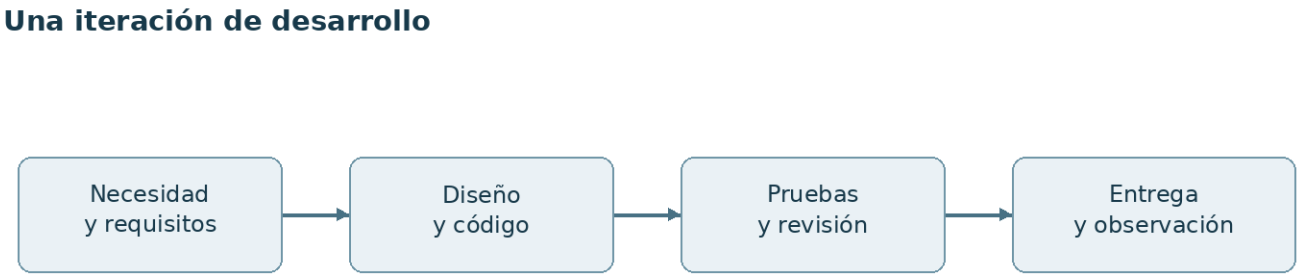


Figura 6. Lo observado en pruebas y uso alimenta la siguiente iteración. La entrega no termina el mantenimiento.

Qué revisar además del resultado

La corrección exige que las salidas correspondan a las entradas y al contrato. La robustez considera errores previsibles. La mantenibilidad facilita entender y cambiar el programa. La portabilidad analiza qué parte depende de un sistema. La seguridad limita lo que el programa puede hacer y evita tratar datos no confiables como instrucciones. La eficiencia busca consumo razonable bajo una carga definida.

La calidad no se demuestra con una única ejecución feliz. Un informe con totales correctos puede estar ocultando errores de permisos; uno muy rápido puede haber omitido la mitad de los archivos. Debes definir qué se midió, con qué datos y qué criterios se cumplieron. La documentación debe coincidir con la versión entregada.

Control de versiones

Conservar cambios permite comparar una versión que funcionaba con otra que falla. Git registra instantáneas del proyecto y su historia. Un repositorio local no equivale a una copia externa ni a un servicio remoto. Para el aula basta empezar con cambios pequeños y mensajes que expliquen la intención; no subas contraseñas, datos personales ni informes reales del centro.

Para practicar

Divide el proyecto en tres incrementos útiles. Para cada uno, describe una prueba y qué documentación cambiaría. Explica cómo recuperarías la última versión conocida si un cambio rompe el cálculo.

25 Proyecto de inventario del laboratorio

Vamos a convertir los requisitos anteriores en una utilidad completa. Consultará una carpeta de prueba y creará un informe CSV en otra ubicación. El ejemplo es intencionadamente limitado: no borra, no renombra, no entra en subcarpetas y no pretende sustituir una herramienta profesional de inventario. Su valor está en que puedes relacionar cada instrucción con un requisito.

Preparar un conjunto de datos reproducible

```
from pathlib import Path

base = Path("datos_demo")
base.mkdir(exist_ok=True)
(base / "a.txt").write_text("hola", encoding="utf-8")
(base / "b.txt").write_text("adios", encoding="utf-8")
(base / "subcarpeta").mkdir(exist_ok=True)
```

Ejecuta la preparación en una carpeta de laboratorio nueva. Crea dos archivos de 4 y 5 bytes respectivamente; no llevan salto de línea. Los textos son ASCII y cada carácter ocupa un byte en UTF-8. La subcarpeta se omite por diseño. Los métodos write_text sustituyen los dos archivos ficticios si ya existen: no cambies sus rutas para apuntar a documentos reales.

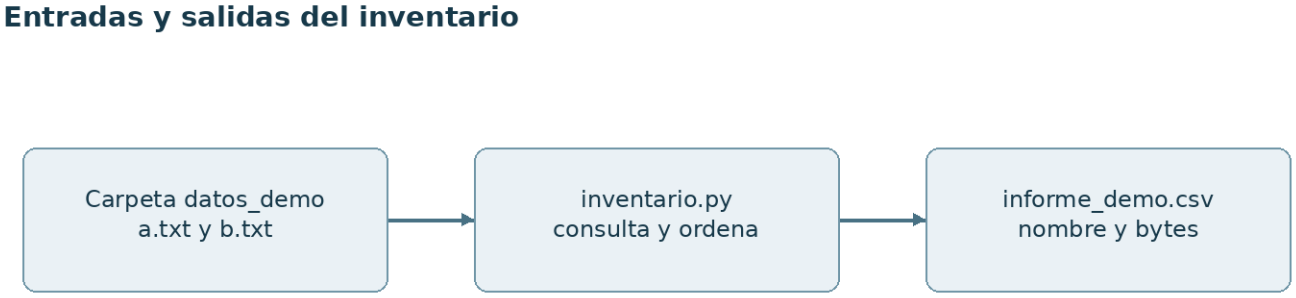


Figura 7. El CSV se guarda fuera de datos_demo. Los archivos originales se consultan, no se alteran.

Qué resultado esperar

Con el conjunto anterior, el programa debe registrar a.txt con 4 bytes y b.txt con 5. El total del resumen será 9 bytes y dos archivos. El CSV incluirá una cabecera y dos registros. No se incluye el tamaño de la carpeta ni se mide el espacio físicamente ocupado en bloques: stat().st_size aporta el tamaño lógico del archivo.

El tamaño lógico y la ocupación real pueden diferir por compresión, archivos dispersos o tamaño de bloque. No sumes este dato y lo presentes como espacio total ocupado del sistema de archivos. Un requisito preciso evita atribuir al resultado una interpretación que el programa no calcula.

26 Código del proyecto y recorrido de archivos

Guarda el código de esta página y de la siguiente en un único archivo llamado inventario.py, conservando el orden. También se facilita en el paquete de ejemplos. La primera parte contiene importaciones y dos funciones; todavía no ejecuta la aplicación principal.

```
"""Inventario didáctico de archivos inmediatos de una carpeta."""
import argparse
import csv
import logging
from pathlib import Path

def inventariar(carpeta):
    filas = []
    omitidos = 0
    for ruta in carpeta.iterdir():
        try:
            if ruta.is_symlink():
                continue
            if ruta.is_file():
                filas.append({
                    "nombre": ruta.name,
                    "bytes": ruta.stat().st_size,
                })
        except OSError as error:
            omitidos += 1
            logging.warning("Consulta omitida: %s",
                            type(error).__name__)
    filas.sort(key=lambda fila: fila["nombre"])
    return filas, omitidos

def guardar_csv(destino, filas):
    with destino.open("x", newline="", encoding="utf-8") as archivo:
        escritor = csv.DictWriter(
            archivo, fieldnames=["nombre", "bytes"])
        escritor.writeheader()
        escritor.writerows(filas)
```

iterdir recorre las entradas inmediatas. continue pasa a la siguiente cuando hay un enlace. is_file selecciona archivos regulares y stat obtiene su tamaño lógico. Algunos errores pueden aparecer antes de llegar al bloque try, al abrir o avanzar el directorio; el programa principal también captura errores de acceso.

La función conserva todas las filas para ordenarlas. Esto hace el resultado reproducible, pero utiliza memoria proporcional al número de archivos. Las subcarpetas y enlaces se omiten por alcance y no cuentan como error. El contador omitidos solo refleja los OSError capturados durante las consultas. Un programa de alta seguridad necesitaría mecanismos adicionales frente a cambios simultáneos y rutas manipuladas.

27 Código del proyecto y programa principal

```
def main(argv=None):
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("carpeta", type=Path)
    parser.add_argument("informe", type=Path)
    args = parser.parse_args(argv)
    logging.basicConfig(level=logging.INFO,
                        format="%(levelname)s: %(message)s")
    try:
        origen = args.carpeta.resolve(strict=True)
        destino = args.informe.resolve()
        if not origen.is_dir():
            raise ValueError("La entrada debe ser una carpeta")
        if destino.parent == origen:
            raise ValueError(
                "Guarda el informe fuera de la carpeta de entrada")
        filas, omitidos = inventariar(origen)
        guardar_csv(destino, filas)
    except (OSError, ValueError) as error:
        logging.error("No se completó el inventario: %s", error)
        return 1
    total = sum(f["bytes"] for f in filas)
    print(f"Archivos: {len(filas)}; bytes: {total}")
    print(f"Consultas omitidas por error: {omitidos}")
    print(f"Informe: {destino}")
    return 2 if omitidos else 0

if __name__ == "__main__":
    raise SystemExit(main())
```

argparse recibe dos argumentos posicionales. resolve(strict=True) exige que la ruta de entrada exista. La comprobación de destino impide crear el informe directamente en la carpeta inventariada. El modo x evita sobrescribir un archivo existente. Si la carpeta de destino no existe o no permite escribir, la operación falla con un mensaje.

SystemExit convierte el resultado de main en código de salida. En esta aplicación, 0 significa finalización sin errores de consulta; 1, fallo; y 2, informe creado con consultas omitidas. argparse también puede terminar con 2 por argumentos inválidos y muestra su propio mensaje antes de inventariar. Esta colisión se documenta; una versión posterior podría reservar códigos distintos para ambas situaciones.

Si la escritura falla después de crear el CSV, puede quedar un archivo parcial. No se anuncia éxito: el código será 1. Antes de reintentar, revisa y retira solo ese informe parcial del laboratorio. Una mejora sería escribir en un temporal y publicar de forma controlada, con una política que conserve la regla de no sobrescritura.

28 Ejecutar y comprobar el proyecto

Primera ejecución

```
python3 inventario.py datos_demo informe_demo.csv
# En Windows, si utilizas el lanzador:
py inventario.py datos_demo informe_demo.csv
```

Elige una de las dos órdenes según el entorno. El informe se crea respecto al directorio de trabajo. Si la ruta contiene espacios, encierra cada argumento en comillas. La ayuda se consulta con python3 inventario.py --help o su alternativa de Windows. No es necesario ejecutar con permisos de administrador.

Salida esperada con los datos de la práctica

```
Archivos: 2; bytes: 9
Consultas omitidas por error: 0
Informe: [ruta absoluta al CSV]
```

El texto entre corchetes representa la ruta que dependa de tu equipo y no una salida literal. Abre el CSV con un editor o una hoja de cálculo y comprueba las columnas. La carpeta datos_demo debe conservar sus dos archivos sin cambios. Al volver a ejecutar con el mismo destino, el modo x debe impedir su sobrescritura.

Caso de prueba	Resultado que debe comprobarse
Dos archivos del ejemplo	2 registros, tamaños 4 y 5, total 9.
Carpeta vacía	Cabecera sin filas y resumen con cero archivos.
Ruta inexistente	Mensaje de error y código 1.
Entrada que es un archivo	Rechazo porque no es una carpeta.
Informe ya existente	Fallo sin sustituir su contenido.
Informe dentro del origen	Rechazo antes del inventario.

Qué significa la finalización parcial

Si un archivo provoca una excepción capturada, se conserva la información obtenida de los demás y el resultado es parcial. No debes interpretar esa salida como «todo correcto». La política se ha elegido para una consulta informativa. Una herramienta que aplicase cambios podría necesitar detenerse al primer fallo para evitar un estado inconsistente.

Para practicar

Añade c.txt con un texto corto, crea otro nombre de informe y predice los nuevos totales. Después verifica el resultado. No reutilices el CSV anterior si la regla exige que el destino sea nuevo.

29 Pruebas automatizadas y depuración

Una prueba automática compara un resultado con una expectativa definida. No reemplaza pensar en los casos, pero permite repetirlos después de cada cambio. Las pruebas unitarias examinan piezas pequeñas; las de integración comprueban colaboraciones, como recorrer una carpeta y crear un CSV; las de aceptación contrastan necesidades del usuario.

```
import tempfile
from pathlib import Path
from inventario import inventariar, guardar_csv

with tempfile.TemporaryDirectory() as temporal:
    base = Path(temporal)
    origen = base / "entrada"
    origen.mkdir()
    (origen / "b.txt").write_bytes(b"12345")
    (origen / "a.txt").write_bytes(b"1234")
    (origen / "sub").mkdir()
    filas, omitidos = inventariar(origen)
    assert [f["nombre"] for f in filas] == ["a.txt", "b.txt"]
    assert sum(f["bytes"] for f in filas) == 9
    assert omitidos == 0
    guardar_csv(base / "salida.csv", filas)
print("Comprobaciones superadas")
```

La carpeta temporal se elimina al abandonar el bloque. Las pruebas no dependen de tus archivos personales ni de un disco lleno. assert sirve aquí para comprobar expectativas durante la prueba, pero no para validar entradas de producción: puede desactivarse al ejecutar Python con optimización. La herramienta utiliza excepciones y decisiones explícitas para sus validaciones.

Depurar sin adivinar

Si el total es 0 en lugar de 9, comprueba primero la carpeta que se recorrió y qué filas se obtuvieron. Si falta un archivo, examina sus características y los mensajes. Si el CSV tiene otro orden, revisa la ordenación. Una prueba que falla señala una diferencia observable; no decide por sí sola dónde está el defecto.

El paquete de ejemplos incluye pruebas ampliadas de no sobrescritura, carpeta vacía, argumentos y rechazo del destino interno. Comprueban resultados y efectos en archivos, no solo que se haya llamado una función. La denegación de permisos depende del entorno y no se fuerza cambiando permisos de carpetas reales del aula.

Para practicar

Diseña una prueba para un nombre de archivo con coma y otra para contenido no ASCII. Predice qué debe conservar el CSV y cómo comprobarías que no se corrompen los nombres.

30 Optimización basada en medidas

Optimizar es reducir un coste relevante manteniendo el comportamiento correcto. Primero establece una medida de referencia. Si el problema es que el informe tarda, mide el tiempo del recorrido, del ordenado y de la escritura. Si es memoria, observa qué estructuras retienen información. No cambies varias cosas a la vez sin poder atribuir el efecto.

```
from time import perf_counter
from pathlib import Path
from inventario import inventariar

inicio = perf_counter()
filas, omitidos = inventariar(Path("datos_demo"))
duracion = perf_counter() - inicio
print(f"Duración observada: {duracion:.6f} segundos")
```

Una carpeta de dos archivos no sirve para afirmar que una versión es más rápida en producción. Ejecuta varias veces con una carga definida, registra el entorno y conserva las medidas. Las cachés, otras aplicaciones y el estado del almacenamiento influyen. perf_counter mide duración transcurrida, no consumo de CPU exclusivamente.

Complejidad y decisiones

Recorrer n entradas exige, como mínimo, examinarlas. Nuestro ordenado añade un coste típico O(n log n), mientras que las filas usan memoria O(n). Si eliminamos el ordenado y escribimos cada registro directamente, podemos reducir memoria, pero cambiamos el contrato de orden reproducible. La mejora debe negociar ese requisito o utilizar otra técnica adecuada a grandes volúmenes.

En Python añadir repetidamente al final de una lista es una operación amortizada eficiente; buscar un código en una lista puede exigir revisar muchos elementos. Un diccionario puede facilitar búsquedas por clave, a costa de memoria y de mantener un modelo correcto. No elijas estructuras solo por una etiqueta de rapidez: comprueba el patrón de acceso.

Cachés y resultados antiguos

Una caché evita repetir un trabajo guardando resultados, pero puede devolver información desactualizada. Si almacenas el espacio libre durante horas, un aviso puede llegar tarde. Define cuándo caduca, cuándo se invalida y qué ocurre si la consulta nueva falla. La exactitud temporal forma parte del requisito del servicio.

Para practicar

Propón una mejora del inventario, describe qué coste reduce y qué prueba garantizaría que no cambia el resultado. Identifica una situación en la que la mejora no aportaría ventaja.

31 Documentar entregar y mantener

Un programa útil debe poder utilizarse y mantenerse sin depender de que su autor esté presente. La documentación incluye finalidad, alcance, requisitos, instalación, ejecución, entradas, salidas, errores y limitaciones. Un comentario explica una decisión no obvia; no es necesario repetir en castellano cada línea de código.

Modelo de ficha de la herramienta

Campo	Ejemplo documentado
Nombre y versión	Inventario de carpeta Aula Norte, versión didáctica 1.0.
Finalidad	Listar nombres y tamaños lógicos de archivos inmediatos.
Requisitos	Python 3.10 o posterior; lectura de la carpeta y creación del informe.
Uso	inventario.py CARPETA INFORME, con rutas adecuadas al equipo.
Salida	CSV UTF-8 con nombre y bytes; resumen por consola.
Límites	Sin recursión; omite enlaces; no ofrece instantánea atómica; conserva filas en memoria.
Errores y códigos	0 correcto; 1 fallo; 2 parcial o error de argumentos según el mensaje.

Entregar una versión reproducible

Incluye el código, instrucciones de ejecución y datos ficticios mínimos. Si existen dependencias externas, registra versiones compatibles y cómo instalarlas; este proyecto usa solo la biblioteca estándar. No confundas empaquetar en un archivo ZIP con instalar un servicio permanente: son operaciones con responsabilidades diferentes.

Preparar cambios con reversión

Para actualizar una herramienta del centro, conserva la versión anterior y la configuración, realiza una copia recuperable si afecta a datos, acuerda una ventana y comunica la intervención. Prueba la versión nueva en el laboratorio. Si no cumple los criterios, vuelve a la versión anterior con el procedimiento previsto y verifica la función relevante.

Una reversión del código no deshace automáticamente cambios de datos. Por eso importa diseñar migraciones, compatibilidad y copias antes de intervenir. Para este proyecto la ejecución no modifica los archivos inventariados, pero una ampliación que los renombre tendría un riesgo distinto y exigiría otra preparación.

Para practicar

Redacta el manual de uso de tu versión en media página. Otra pareja debe poder ejecutarla con datos ficticios sin preguntarte qué argumentos necesita.

32 Objetos componentes y siguientes unidades

Un objeto reúne estado y operaciones. Una clase define cómo se construyen y se comportan determinados objetos. Esta forma de organizar código puede ayudar cuando varias operaciones comparten datos y reglas, pero no obliga a convertir cada función pequeña en una clase. Lo importante es que la responsabilidad quede clara.

```
from dataclasses import dataclass

@dataclass
class MedidaDisco:
    total: int
    libre: int

    def porcentaje_libre(self):
        if self.total <= 0 or not 0 <= self.libre <= self.total:
            raise ValueError("Medida inválida")
        return self.libre / self.total * 100

medida = MedidaDisco(total=200, libre=30)
print(medida.porcentaje_libre())
```

dataclass genera parte del código repetitivo de inicialización. self representa la instancia sobre la que se invoca el método. Las anotaciones int documentan el tipo esperado, pero no lo validan automáticamente en tiempo de ejecución. El ejemplo calcula 15.0 a partir de la misma regla usada antes.

De una función a un componente

Un componente ofrece capacidades mediante interfaces y tiene dependencias, configuración y condiciones de uso. No cualquier archivo de código constituye por sí solo un componente listo para integrar. Deben estar definidos el contrato, los errores, la versión y cómo se incorpora o despliega. Una interfaz estable permite cambiar la implementación sin obligar a rehacer todos los consumidores.

UF1287 profundizará en el manejo de dispositivos y sus controladores. UF1288 aborda servicios de comunicaciones: además de enviar datos habrá que considerar protocolos, concurrencia, autenticación y fallos de red. UF1289, UF1290 y UF1291 desarrollan diseño, integración y despliegue por componentes. Este capítulo prepara vocabulario, no sustituye esos contenidos.

Para practicar

Si otra aplicación quiere consumir tu inventario, ¿qué conviene ofrecer: texto libre, una función con retorno estructurado o un archivo con esquema documentado? Justifica tu elección según el consumidor.

33 Prácticas progresivas para entregar

Las siguientes prácticas no incluyen soluciones. Para cada una entrega código, datos ficticios, pruebas y una explicación breve. Antes de ejecutar, describe entradas, salidas y criterios de aceptación. Trabaja siempre en una carpeta de laboratorio y conserva los originales.

Práctica 1 Clasificar medidas

Recibe capacidad y espacio libre; valida las entradas y calcula el porcentaje. Clasifica en CRITICO por debajo de 5 %, AVISO desde 5 % hasta menos de 15 % y OK desde 15 %. Prueba límites y entradas inválidas. Muestra unidades y evita dividir entre cero. Separa cálculo y presentación.

Práctica 2 Inventario lógico del aula

Representa cinco equipos ficticios con código, sistema operativo y RAM en GiB. Identifica los que tienen menos de 8 GiB y los datos no verificados. Exporta los resultados a CSV. No inventes valores reales del aula. Justifica el tratamiento de una capacidad desconocida y comprueba códigos repetidos.

Práctica 3 Analizar un registro

Crea un registro ficticio con fecha, nivel y mensaje. Cuenta eventos por nivel leyendo por líneas y define cómo tratar líneas incompletas. Documenta su formato y aporta al menos una línea inválida. No interpretes una palabra ERROR dentro del mensaje como nivel sin comprobar el campo correspondiente.

Práctica 4 Ampliar el proyecto

Añade al inventario un filtro de extensión configurable y un mínimo de tamaño en bytes. Especifica cómo se trata la diferencia entre mayúsculas y minúsculas. Conserva la regla de no sobrescribir y el informe de omisiones. Comprueba que un filtro sin coincidencias produce un informe vacío válido, no un fallo.

Práctica 5 Revisión entre parejas

Intercambia la herramienta con otra pareja. Ejecutadla con los datos proporcionados, comprobad el contrato y anotad dos mejoras justificadas. No cambiéis la versión del compañero sin acordarlo. Presentad un registro con caso, resultado esperado, resultado observado y conclusión.

Reto de ampliación

Diseña una versión que gestione grandes carpetas con menor memoria. Explica si mantiene el orden de salida y cómo lo consigue. No basta con añadir hilos: aporta medidas, límites y pruebas de equivalencia. Puedes entregar un diseño razonado antes de implementar.

34 Guía de revisión y autoevaluación

Comprender antes de escribir

Explica con tus palabras la diferencia entre algoritmo, programa y proceso. Identifica qué datos entran y qué resultado se espera. Señala qué parte consulta el sistema y cuál calcula. Si no puedes predecir una ejecución con tres datos sencillos, reduce el problema y realiza una traza antes de añadir más código.

Revisar los límites

Comprueba cero elementos, un elemento y varios; valores justo antes, en y después del umbral; rutas inexistentes y destinos ocupados. Si la herramienta recibe texto externo, revisa conversión y validación. Anota qué ocurrió realmente, no solo «funciona». Un resultado correcto con un dato no demuestra que se cumplen todas las reglas.

Pregunta de revisión	Evidencia que debes aportar
¿Calcula lo correcto?	Casos con resultado previsto y una traza explicada.
¿Informa de fallos?	Una ejecución inválida y su mensaje o código.
¿Respeta los originales?	Comprobación de archivos y ausencia de cambios no previstos.
¿Se puede reutilizar?	Funciones con contrato y ejemplos de llamada.
¿Otra persona puede ejecutarlo?	Instrucciones, versión y datos ficticios.
¿La mejora es demostrable?	Medidas comparables y pruebas que mantienen el resultado.

Errores frecuentes que debes reconocer

Confundir = con ==; olvidar que range excluye su límite final; reinicializar un acumulador dentro del bucle; sumar texto sin convertir; mezclar bytes con GiB; tratar falta de permiso como ausencia del archivo; guardar el informe dentro de su propio origen; sobrescribir por usar w sin revisar el requisito; capturar cualquier excepción y continuar como si nada.

Una explicación final de un minuto

Presenta tu herramienta indicando qué problema resuelve, qué recibe, qué produce, qué no hace y cómo sabes que cumple. Menciona una limitación real y una mejora futura. Este ejercicio comprueba comprensión y evita que la entrega sea solo un archivo que otra persona no puede interpretar.

Si necesitas refuerzo, vuelve a variables, condiciones y trazas con valores pequeños. Si avanzas con soltura, mejora pruebas, documentación y eficiencia antes de aumentar funcionalidades. Un programa sencillo bien comprobado proporciona una base más útil que muchas opciones cuyo comportamiento no puedes explicar.

35 Glosario de consulta

Algoritmo. Secuencia ordenada y finita de pasos para resolver un problema.

API. Interfaz que define operaciones disponibles y las reglas para utilizarlas.

Biblioteca. Código reutilizable que proporciona funciones, clases u otros recursos.

Compilador. Programa que traduce código de una representación a otra.

Concurrencia. Progreso de varias tareas con intervalos de actividad solapados.

Contrato. Condiciones de entrada, resultado y fallos que una pieza de software declara.

Excepción. Señal de una situación que altera el flujo normal de ejecución.

Interfaz. Punto de interacción entre partes con operaciones y formatos definidos.

Memoria virtual. Mecanismos que ofrecen y gestionan espacios de direcciones de procesos.

Módulo. Unidad de organización de código; en Python, habitualmente un archivo importable.

Núcleo. Parte privilegiada del sistema que gestiona mecanismos esenciales.

Persistencia. Conservación de información más allá de una ejecución o pérdida de alimentación, según el soporte y las garantías.

Proceso. Instancia en ejecución de un programa, con recursos y estado.

Puntero. Valor que representa una dirección utilizada para acceder a un objeto según su tipo y validez.

Requisito. Capacidad o condición que debe cumplir la solución y que necesita un criterio de comprobación.

Serialización. Conversión de datos a una representación almacenable o transmisible.

Traza. Seguimiento de valores y pasos durante una ejecución real o simulada.

Validación. Comprobación de que los datos cumplen formato, tipos y reglas del problema.

36 Fuentes y correspondencia de contenidos

Manual de elaboración original para el aula. Los ejemplos y las ilustraciones se han creado para este material; no reproducen un manual comercial. El enfoque formativo utiliza la ficha IFCT0609 aportada por el docente y contrasta conceptos técnicos con documentación oficial. Consulta realizada el 29 de octubre de 2026.

Correspondencia orientativa con UF1286

Eje de contenidos	Capítulos del manual
Servicios y estructura de sistemas operativos	2 a 4 y 17 a 22.
Lenguajes bibliotecas y servicios del sistema	5, 12 a 18 y 21.
Requisitos y ciclo de vida	1, 23, 24 y 31.
Desarrollo calidad y optimización	6 a 16 y 25 a 30.
Puente hacia otras unidades	32 y actividades de ampliación.

La correspondencia es didáctica y no constituye una acreditación de cobertura completa de todos los criterios oficiales. El docente debe completar, secuenciar y evaluar los contenidos según la programación del curso y el entorno utilizado.

Documentos de referencia

Ficha IFCT0609

Documento aportado al curso: Programación de sistemas informáticos, módulos y unidades formativas.

Referencia curricular

Real Decreto 686/2011, anexo VI, y modificaciones aplicables. https://www.boe.es/diario_boe/txt.php?id=BOE-A-2011-10055

Python Tutorial

Fundamentos del lenguaje. https://docs.python.org/3.12/tutorial/

Biblioteca estándar

Documentación de rutas, procesos y operaciones de archivos. https://docs.python.org/3.12/library/pathlib.html
https://docs.python.org/3.12/library/subprocess.html
https://docs.python.org/3.12/library/shutil.html

Datos y registros

https://docs.python.org/3.12/library/csv.html
https://docs.python.org/3.12/library/json.html
https://docs.python.org/3.12/library/logging.html

Las URLs de documentación se fijan en la rama 3.12 para facilitar una referencia reproducible; no implican que se deba instalar una versión concreta sin seguir las indicaciones del centro. Las páginas pueden recibir correcciones con el tiempo.